

(Very) Basics of Nextflow

Diego Garrido Martín

*UST, University of Vic, Vic
Computational Biology of RNA Processing, CRG, Barcelona*



- Challenges in bioinformatics pipelines
- What is Nextflow
- Integration with Git & Docker
- Use case: ChIP-seq pipeline

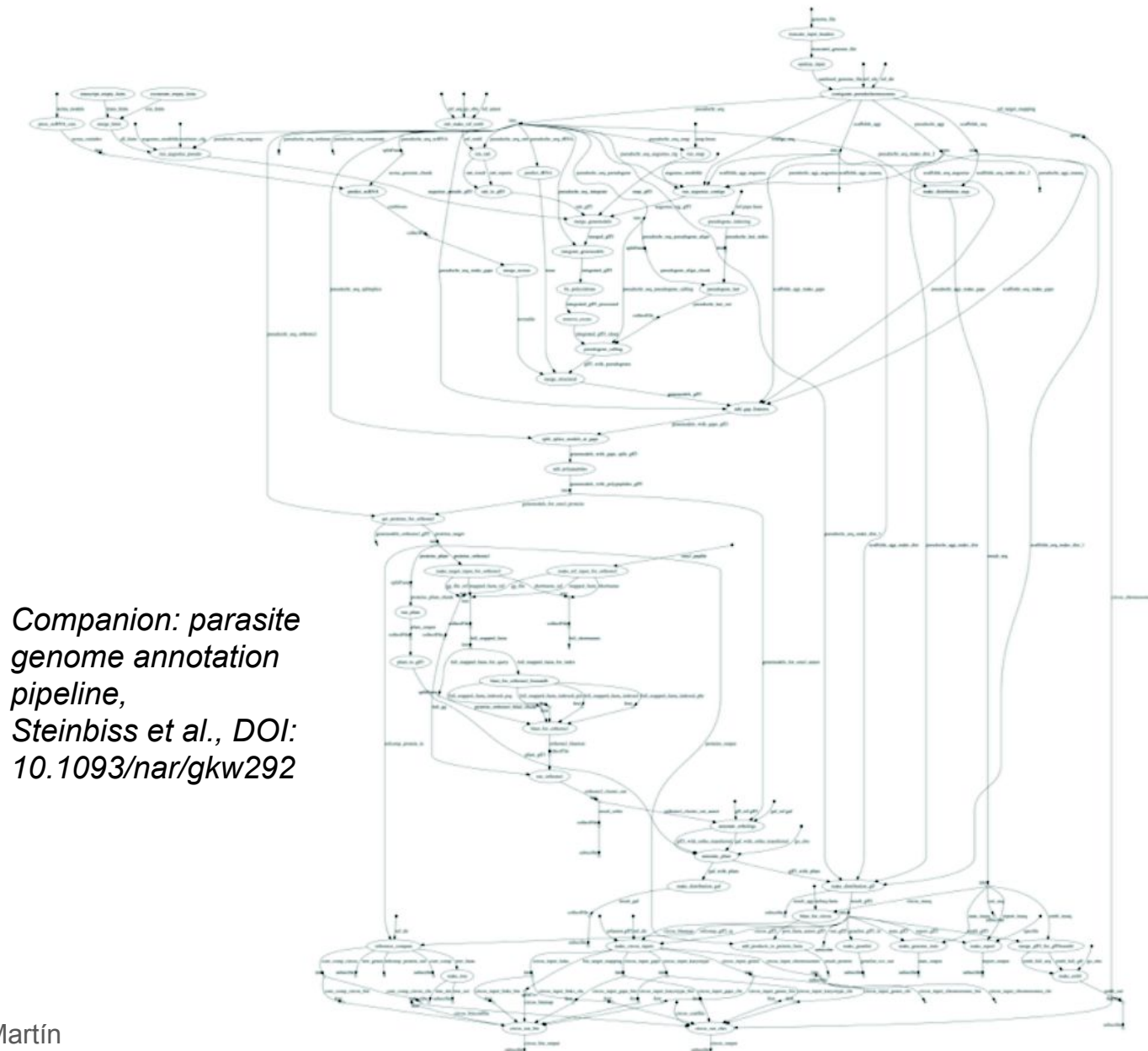


Bioinformatic pipelines

```
#!/bin/bash
```

```
blastp -query sample.fasta -outfmt 6 \  
| head -n 10 \  
| cut -f 2 \  
| blastdbcmd -entry - > sequences.txt
```

Bioinformatic pipelines



Challenges

- Parallelization
- Portability
- Scalability
- Reproducibility
- Error handling

- **Nextflow** defines a Domain Specific Language (DSL) for computational pipelines
- It's a superset of **Groovy** programming language
- Three main abstractions:
 - **process**: executes user code
 - **channels**: connects processes
 - **operators**: transforms channel contents

Fast prototyping

Nextflow allows you to write a computational pipeline by making it simpler to put together many different tasks.

You may reuse your existing scripts and tools and you don't need to learn a new language or API to start using it.

Portable

Nextflow provides an abstraction layer between your pipeline's logic and the execution layer, so that it can be executed on multiple platforms without it changing.

It provides out of the box executors for SGE, LSF, SLURM, PBS and HTCondor batch schedulers and for [Kubernetes](#), [Amazon AWS](#) and [Google Cloud](#) platforms.

Continuous checkpoints

All the intermediate results produced during the pipeline execution are automatically tracked.

This allows you to resume its execution, from the last successfully executed step, no matter what the reason was for it stopping.

Reproducibility

Nextflow supports [Docker](#) and [Singularity](#) containers technology.

This, along with the integration of the [GitHub](#) code sharing platform, allows you to write self-contained pipelines, manage versions and to rapidly reproduce any former configuration.

Unified parallelism

Nextflow is based on the *dataflow* programming model which greatly simplifies writing complex distributed pipelines.

Parallelisation is implicitly defined by the processes input and output declarations. The resulting applications are inherently parallel and can scale-up or scale-out, transparently, without having to adapt to a specific platform architecture.

Stream oriented

Nextflow extends the Unix pipes model with a fluent DSL, allowing you to handle complex stream interactions easily.

It promotes a programming approach, based on functional composition, that results in resilient and easily reproducible pipelines.

How does a script look like?

```
sequences = Channel.fromPath("/data/sample.fasta")
```

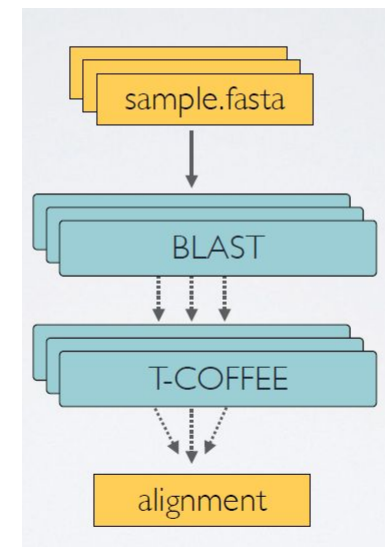
```
process blast {  
  input:  
  file 'in.fasta' from sequences  
  output:  
  file 'out.txt' into blast_result  
  
  """  
  blastp -query in.fasta -outfmt 6 | cut -f 2 | \  
  blastdbcmd -entry_batch - > out.txt  
  """  
}
```

```
process align {  
  input:  
  file all_seqs from blast_result  
  output:  
  file align_result  
  
  """  
  t_coffee $all_seqs 2>&- | tee align_result  
  """  
}
```

```
blast_result.collectFile(name: 'final_alignment')
```

Implicit parallelism

```
Channel.fromPath("/data/*.fasta")
```



```
process {  
  executor = 'sge'  
  queue = 'rg-el6'  
  memory = '10G'  
  cpus = 8  
  time = '2h'
```

```
}
```

Easy config

Integration with Git & Docker

- Easy installation
- Use existing tools and scripts
- Implicit parallelisation
- Simplified deployment
- Lightweight, self-contained

Easiness

nextflow

containers



HPC

clusters and cloud

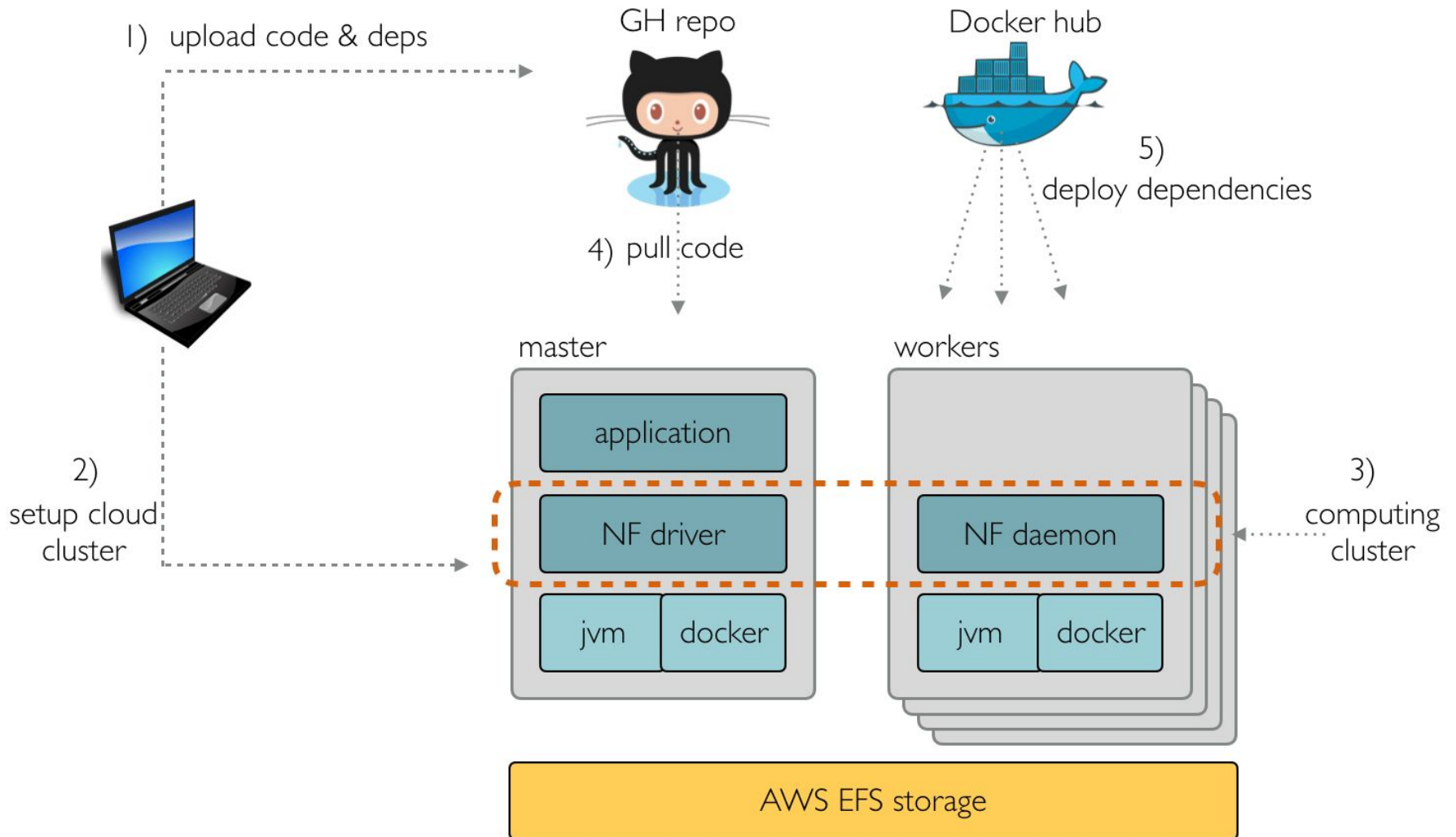


Reproducibility

versioning



Integration with Git & Docker



Use case: chip-nf

<https://github.com/guigolab/chip-nf>

Additional resources

- Nextflow documentation
 - <https://www.nextflow.io/docs/latest/index.html>
- More examples of NF pipelines
 - <https://github.com/nextflow-io/awesome-nextflow>